AEROSTACK2
Dynamic Multi-Drone Area Survey and Task Reallocation

ROS 2 Humble and Gazebo Based Multi-Drone Autonomous Survey with Dynamic Work Transfer and Mission Recovery

Source file: area_survey.py
Complete source captured from the project terminal and included for mentor reference.

```
#!/usr/bin/env python3

import time
import threading
import rclpy

from as2_python_api.drone_interface import DroneInterface


# CONFIGURATION

TAKEOFF_HEIGHT = 1.0
TAKEOFF_SPEED = 0.7
LAND_SPEED = 0.4
MISSION_SPEED = 1.0

STABILIZATION_TIME = 3.0

# Simulated delayed-drone condition
SIMULATED_DELAY = 8.0
DELAY_DRONE = "drone0"

# Zero-based index:
# 0 = waypoint 1
# 1 = waypoint 2
# 2 = waypoint 3
# 3 = waypoint 4
DELAY_WAYPOINT_INDEX = 2

# Time to wait for a naturally idle helper.
HELPER_WAIT_TIMEOUT = 5.0

# Maximum time allowed for a stopped helper thread to exit.
PREEMPT_STOP_TIMEOUT = 8.0

# Maximum time allowed for one asynchronous GoTo
GOTO_TIMEOUT = 60.0


# SURVEY AREAS

AREA_A = [
    (-2.5, -2.0, 1.0),
    (-2.5,  2.0, 1.0),
    (-1.0,  2.0, 1.0),
    (-1.0, -2.0, 1.0),
]

AREA_B = [
    (-0.5, -2.0, 1.0),
    (-0.5,  2.0, 1.0),
    ( 0.5,  2.0, 1.0),
    ( 0.5, -2.0, 1.0),
]

AREA_C = [
    (1.0, -2.0, 1.0),
    (1.0,  2.0, 1.0),
    (2.5,  2.0, 1.0),
    (2.5, -2.0, 1.0),
]

AREAS = {
    "A": AREA_A,
    "B": AREA_B,
    "C": AREA_C,
}

INITIAL_ASSIGNMENT = {
    "drone0": "A",
    "drone1": "B",
```

```
    "drone2": "C",
}


# DRONE CLASS

class SurveyDrone(DroneInterface):

    def __init__(self, namespace):

        super().__init__(
            namespace,
            verbose=True,
            use_sim_time=True
        )

        self.name = namespace

        # Mission state
        self.mission_state = "READY"

        # Original assigned area
        self.original_area = None
        self.original_waypoints = []

        # Waypoint progress for original area
        self.completed_indices = set()
        self.current_index = None

        # Preemption control
        self.preempt_requested = False

        # Thread reference
        self.thread = None

        # Drone0 delay detection
        self.delay_event = threading.Event()
        self.reassignment_event = threading.Event()

        # Thread safety
        self.lock = threading.Lock()



    # STATE

    def set_state(self, state):

        with self.lock:
            self.mission_state = state


    def get_state(self):

        with self.lock:
            return self.mission_state


    def get_completed_indices(self):

        with self.lock:
            return set(self.completed_indices)


    def get_remaining_indices(self):

        with self.lock:

            completed = set(self.completed_indices)

            return [
                i
                for i in range(len(self.original_waypoints))
                if i not in completed
            ]
```

```
# GO TO

def execute_go_to(self, point):

    x, y, z = point

    print(
        f"{self.name}: GO_TO "
        f"({x:.2f}, {y:.2f}, {z:.2f})"
    )

    try:

        # Do not start a new GoTo while another one is active.
        if self.go_to.is_running():

            print(
                f"{self.name}: ERROR - GoTo handler is still running"
            )

            return False


        # ASYNCHRONOUS GOTO

        accepted = self.go_to(
            x,
            y,
            z,
            MISSION_SPEED,
            0,
            None,
            "earth",
            False
        )

        if not accepted:

            print(
                f"{self.name}: GoTo goal was not accepted"
            )

            return False

        start_time = time.monotonic()

        # Wait for the asynchronous behavior to finish.
        while self.go_to.is_running():

            # Safety timeout.
            if (
                time.monotonic() - start_time
                > GOTO_TIMEOUT
            ):

                print(
                    f"{self.name}: GoTo timeout after "
                    f"{GOTO_TIMEOUT:.1f}s"
                )

                try:
                    self.go_to.stop()
                except Exception:
                    pass

                return False

            time.sleep(0.1)

        # If the behavior ended because this drone was
        # preempted, this waypoint was NOT completed.
        with self.lock:
            was_preempted = self.preempt_requested
```

```
        if was_preempted:
            return False

        print(
            f"{self.name}: GoTo behavior finished"
        )

        return True

    except Exception as exc:

        print(
            f"{self.name}: GoTo exception: {exc}"
        )

        return False


# STOP CURRENT BEHAVIOR


def stop_current_behavior(self):

    try:

        if self.go_to.is_running():

            print(
                f"{self.name}: stopping current GoTo behavior..."
            )

            result = self.go_to.stop()

            print(
                f"{self.name}: GoTo stop result = {result}"
            )

            return bool(result)

        else:

            print(
                f"{self.name}: no active GoTo behavior"
            )

            return True

    except Exception as exc:

        print(
            f"{self.name}: stop exception: {exc}"
        )

        return False



# INITIAL SURVEY THREAD

def run_initial_survey(
    self,
    manager,
    area_name,
    waypoints,
    delay_enabled=False
):

    self.original_area = area_name
    self.original_waypoints = list(waypoints)

    self.set_state("SURVEYING")

    print()
    print("=" * 60)
    print(
        f"{self.name}: STARTING AREA {area_name}"
```

```
)
print("=" * 60)

for i, point in enumerate(waypoints):


    # Check preemption before starting next waypoint

    with self.lock:
        if self.preempt_requested:

            self.mission_state = "PREEMPTED"

            print(
                f"{self.name}: PREEMPTED "
                f"before waypoint {i + 1}"
            )

            return


        self.current_index = i



    # Simulated Drone0 delay

    if delay_enabled and i == DELAY_WAYPOINT_INDEX:

        self.set_state("DELAYED")

        print()
        print("=" * 60)
        print("SIMULATED DELAY")
        print("=" * 60)

        print(
            f"{self.name}: DELAY detected before "
            f"waypoint {i + 1}"
        )

        print(
            f"Simulated delay condition = "
            f"{SIMULATED_DELAY:.1f} seconds"
        )

        print(
            f"{self.name}: waiting for "
            f"task reassignment..."
        )

        # Tell mission manager immediately.
        self.delay_event.set()

        # Wait until mission manager transfers
        # the remaining task.
        while not self.reassignment_event.wait(0.2):

            if self.get_state() == "FAILED":
                return

        self.set_state("TRANSFERRED")

        print(
            f"{self.name}: remaining work "
            f"transferred to helper"
        )

        return
```

```
    # Execute waypoint

    success = self.execute_go_to(point)


    with self.lock:

        was_preempted = self.preempt_requested

        if success:

            self.completed_indices.add(i)
            manager.mark_complete(area_name, i)


    if success:

        print(
            f"{self.name}: waypoint {i + 1} "
            f"COMPLETED"
        )

    else:

        if was_preempted:

            self.set_state("PREEMPTED")

            print(
                f"{self.name}: waypoint {i + 1} "
                f"interrupted by preemption"
            )

            return

        else:

            self.set_state("FAILED")

            print(
                f"{self.name}: waypoint {i + 1} FAILED"
            )

            return


    # If stop happened immediately after GoTo success


    with self.lock:

        if self.preempt_requested:

            self.mission_state = "PREEMPTED"

            print(
                f"{self.name}: preemption detected "
                f"after waypoint {i + 1}"
            )

            return



# Entire original area completed

self.set_state("AVAILABLE")

print()
print(
    f"{self.name}: AREA {area_name} "
    f"COMPLETED"
)
```

```
# MISSION MANAGER

class AreaSurveyManager:

    def __init__(self):

        self.drones = {}

        self.completed_work = set()

        self.progress_lock = threading.Lock()

        self.reallocation_done = False

        self.reallocation_helper = None

        self.reallocation_record = None


    # MARK GLOBAL WAYPOINT COMPLETE


    def mark_complete(self, area_name, index):

        with self.progress_lock:

            self.completed_work.add(
                (area_name, index)
            )


    # CHECK AREA PROGRESS

    def area_progress(self, area_name):

        with self.progress_lock:

            return sum(
                1
                for area, _ in self.completed_work
                if area == area_name
            )


    # EXECUTE A TASK ON A DRONE

    def execute_reassigned_task(
        self,
        drone,
        area_name,
        waypoints,
        indices,
        task_description
    ):

        print()
        print("=" * 60)
        print(task_description)
        print("=" * 60)

        if not indices:

            print(
                f"{drone.name}: no remaining waypoints"
            )

            return True


        drone.set_state("REASSIGNED")
```

```
    for index in indices:

        point = waypoints[index]

        print()
        print(
            f"{drone.name}: assigned "
            f"Area {area_name} waypoint "
            f"{index + 1}/{len(waypoints)}"
        )

        success = drone.execute_go_to(point)

        if not success:

            print()
            print(
                f"{drone.name}: FAILED reassigned "
                f"waypoint {index + 1}"
            )

            drone.set_state("FAILED")

            return False


        self.mark_complete(
            area_name,
            index
        )

        print(
            f"{drone.name}: Area {area_name} "
            f"waypoint {index + 1} COMPLETED"
        )


    return True


# FIND NATURALLY AVAILABLE HELPER

def find_naturally_available_helper(self):

    helpers = [
        self.drones["drone1"],
        self.drones["drone2"],
    ]

    for helper in helpers:

        if helper.get_state() == "AVAILABLE":

            return helper

    return None


# SELECT BUSY HELPER FOR PREEMPTION

def select_busy_helper(self):

    candidates = []

    for name in ["drone2", "drone1"]:

        drone = self.drones[name]

        state = drone.get_state()

        if state == "SURVEYING":

            candidates.append(drone)
```

```
    if not candidates:

        return None


    # Prefer the drone that has already completed
    # more of its current survey.
    candidates.sort(
        key=lambda d: len(d.get_completed_indices()),
        reverse=True
    )

    return candidates[0]



# PREEMPT HELPER

def preempt_helper(self, helper):

    print()
    print("=" * 60)
    print("PREEMPTING BUSY HELPER")
    print("=" * 60)

    print(
        f"Selected helper: {helper.name}"
    )

    print(
        f"Current state: {helper.get_state()}"
    )

    print(
        f"Original area: "
        f"{helper.original_area}"
    )

    print(
        f"Completed original waypoints: "
        f"{sorted(helper.get_completed_indices())}"
    )


    # Tell worker thread not to start another
    # waypoint after the current GoTo ends.
    with helper.lock:
        helper.preempt_requested = True


    # Stop current GoTo.
    stop_result = helper.stop_current_behavior()

    print(
        f"{helper.name}: stop request = "
        f"{stop_result}"
    )


    # Wait for the old survey thread to exit.
    if helper.thread is not None:

        helper.thread.join(
            timeout=PREEMPT_STOP_TIMEOUT
        )


    if (
        helper.thread is not None
        and helper.thread.is_alive()
    ):

        print()
        print(
            f"ERROR: {helper.name} survey thread "
```

```
            f"did not stop safely."
        )

        return None


    remaining = helper.get_remaining_indices()

    print()
    print(
        f"{helper.name}: original survey "
        f"successfully preempted"
    )

    print(
        f"{helper.name}: unfinished original "
        f"waypoints = "
        f"{[i + 1 for i in remaining]}"
    )


    helper.set_state("PREEMPTED")

    # The old survey worker has completely finished.
    # Clear the flag before giving this drone a new task.
    with helper.lock:
        helper.preempt_requested = False

    # Give the action server a short time to settle.
    time.sleep(0.75)

    return remaining



# WAIT FOR HELPER

def wait_for_helper(self):

    print()
    print("=" * 60)
    print("SEARCHING FOR HELPER DRONE")
    print("=" * 60)

    start = time.monotonic()

    while (
        time.monotonic() - start
        < HELPER_WAIT_TIMEOUT
    ):

        helper = (
            self.find_naturally_available_helper()
        )

        if helper is not None:

            print(
                f"Available helper found: "
                f"{helper.name}"
            )

            return helper


        elapsed = (
            time.monotonic() - start
        )

        remaining = max(
            0.0,
            HELPER_WAIT_TIMEOUT - elapsed
        )

        print(
            "No idle helper yet. "
```

```
            f"Waiting {remaining:.1f}s..."
        )

        time.sleep(1.0)


    print()
    print(
        "No helper became naturally available."
    )

    print(
        "Switching to PREEMPTIVE REALLOCATION."
    )

    return self.select_busy_helper()


# DYNAMIC REALLOCATION

def perform_reallocation(self):

    drone0 = self.drones["drone0"]

    remaining_a = drone0.get_remaining_indices()

    print()
    print("=" * 60)
    print("DYNAMIC TASK REALLOCATION")
    print("=" * 60)

    print(
        "Drone0 = DELAYED"
    )

    print(
        f"Area A completed: "
        f"{len(drone0.get_completed_indices())}/"
        f"{len(AREA_A)}"
    )

    print(
        "Drone0 remaining waypoints:"
    )

    for index in remaining_a:

        point = AREA_A[index]

        print(
            f"  A{index + 1}: "
            f"({point[0]:.2f}, "
            f"{point[1]:.2f}, "
            f"{point[2]:.2f})"
        )


    if not remaining_a:

        print(
            "Drone0 has no remaining work."
        )

        drone0.reassignment_event.set()

        return True



    # Find helper

    helper = self.wait_for_helper()

    if helper is None:
```

```
    print()
    print(
        "ERROR: No helper drone available "
        "for reallocation."
    )

    return False



helper_remaining = []

if helper.get_state() == "SURVEYING":

    helper_remaining = (
        self.preempt_helper(helper)
    )

    if helper_remaining is None:

        return False


elif helper.get_state() == "AVAILABLE":

    helper_remaining = (
        helper.get_remaining_indices()
    )


else:

    print(
        f"ERROR: Helper {helper.name} "
        f"is in state {helper.get_state()}"
    )

    return False



self.reallocation_helper = helper.name

self.reallocation_record = {
    "source_drone": "drone0",
    "helper_drone": helper.name,
    "area": "A",
    "waypoints": list(remaining_a),
}

print()
print("=" * 60)
print("TASK TRANSFER")
print("=" * 60)

print(
    f"Drone0 -> {helper.name}"
)

print(
    f"Transferred Area A waypoints: "
    f"{[i + 1 for i in remaining_a]}"
)


success = self.execute_reassigned_task(
    helper,
    "A",
    AREA_A,
    remaining_a,
    f"{helper.name}: EXECUTING DRONE0 REMAINING WORK"
)

if not success:

    print(
```

```
            "ERROR: Reassigned Area A task failed."
        )

        return False


print()
print("=" * 60)
print("DRONE0 WORK COMPLETED BY HELPER")
print("=" * 60)

print(
    f"Area A progress: "
    f"{self.area_progress('A')}/4"
)


# Tell Drone0 its remaining task has been transferred.
drone0.reassignment_event.set()



# Resume helper's original area.

if helper_remaining:

    print()
    print("=" * 60)
    print(
        f"RESUMING {helper.name} ORIGINAL SURVEY"
    )
    print("=" * 60)

    print(
        f"Original area: "
        f"{helper.original_area}"
    )

    print(
        "Remaining original waypoints: "
        f"{[i + 1 for i in helper_remaining]}"
    )


    with helper.lock:
        helper.preempt_requested = False


    success = self.execute_reassigned_task(
        helper,
        helper.original_area,
        helper.original_waypoints,
        helper_remaining,
        f"{helper.name}: RESUMING ORIGINAL SURVEY"
    )

    if not success:

        print(
            f"ERROR: {helper.name} could not "
            f"complete its original survey."
        )

        return False


else:

    with helper.lock:
        helper.preempt_requested = False


helper.set_state("AVAILABLE")

self.reallocation_done = True
```

```
    print()
    print("=" * 60)
    print("REALLOCATION COMPLETE")
    print("=" * 60)

    print(
        f"{helper.name} completed Drone0's "
        f"remaining Area A work."
    )

    if helper.original_area:

        print(
            f"{helper.name} also completed "
            f"its original Area "
            f"{helper.original_area} work."
        )

    return True


# INITIAL TAKEOFF

def takeoff_all(self):

    print()
    print("=" * 60)
    print("TAKING OFF ALL DRONES")
    print("=" * 60)


    results = {}
    threads = []


    def takeoff(drone):

        try:

            print(
                f"{drone.name}: ARMING..."
            )

            drone.arm()

            print(
                f"{drone.name}: OFFBOARD..."
            )

            drone.offboard()

            print(
                f"{drone.name}: TAKING OFF..."
            )

            result = drone.takeoff(
                TAKEOFF_HEIGHT,
                TAKEOFF_SPEED
            )

            results[drone.name] = bool(result)

            print(
                f"{drone.name}: TAKEOFF "
                f"{'SUCCESS' if result else 'FAILED'}"
            )

        except Exception as exc:

            print(
                f"{drone.name}: takeoff error: "
                f"{exc}"
            )

            results[drone.name] = False
```

```
    for drone in self.drones.values():

        thread = threading.Thread(
            target=takeoff,
            args=(drone,),
            daemon=True
        )

        threads.append(thread)
        thread.start()


    for thread in threads:
        thread.join()


    if not all(results.values()):

        print()
        print(
            "ERROR: One or more drones "
            "failed to take off."
        )

        return False


    print()
    print(
        "All drones airborne."
    )

    print(
        f"Stabilizing for "
        f"{STABILIZATION_TIME:.1f} seconds..."
    )

    time.sleep(
        STABILIZATION_TIME
    )

    return True



# START INITIAL SURVEYS

def start_initial_surveys(self):

    print()
    print("=" * 60)
    print("STARTING PARALLEL AREA SURVEY")
    print("=" * 60)

    for drone_name, area_name in INITIAL_ASSIGNMENT.items():

        drone = self.drones[drone_name]

        waypoints = AREAS[area_name]

        delay_enabled = (
            drone_name == DELAY_DRONE
        )

        print(
            f"{drone_name} -> Area {area_name}"
        )

        drone.thread = threading.Thread(
            target=drone.run_initial_survey,
            args=(
                self,
                area_name,
                waypoints,
```

```
            delay_enabled
        ),
        daemon=True
    )

    drone.thread.start()


# WAIT FOR DELAY


def wait_for_delay_detection(self):

    print()
    print("=" * 60)
    print("MISSION MANAGER ACTIVE")
    print("=" * 60)

    print(
        f"Waiting for {DELAY_DRONE} "
        f"delay detection..."
    )

    drone0 = self.drones["drone0"]

    while not drone0.delay_event.wait(0.5):

        if drone0.get_state() == "FAILED":

            print(
                "ERROR: Drone0 failed before "
                "delay condition."
            )

            return False


    print()
    print(
        "MISSION MANAGER: DELAY DETECTED"
    )

    return True



def wait_for_remaining_threads(self):

    print()
    print("=" * 60)
    print("WAITING FOR SURVEY THREADS")
    print("=" * 60)

    for name, drone in self.drones.items():

        if drone.thread is None:
            continue

        # Helper thread was already joined during
        # preemption.
        if (
            self.reallocation_helper == name
            and not drone.thread.is_alive()
        ):

            continue


        drone.thread.join()


        print(
            f"{name}: thread completed "
            f"with state "
```

```
            f"{drone.get_state()}"
        )


# FINAL VERIFICATION

def verify_mission(self):

    print()
    print("=" * 60)
    print("FINAL MISSION VERIFICATION")
    print("=" * 60)


    all_complete = True


    for area_name, waypoints in AREAS.items():

        completed = self.area_progress(
            area_name
        )

        total = len(waypoints)

        print(
            f"Area {area_name}: "
            f"{completed}/{total}"
        )

        if completed != total:

            all_complete = False

            missing = []

            for i in range(total):

                if (
                    area_name,
                    i
                ) not in self.completed_work:

                    missing.append(
                        i + 1
                    )

            print(
                f"  Missing waypoints: "
                f"{missing}"
            )


    print()

    if self.reallocation_done:

        print(
            "Dynamic reallocation: COMPLETED"
        )

        print(
            f"Helper drone: "
            f"{self.reallocation_helper}"
        )

    else:

        print(
            "Dynamic reallocation: "
            "NOT REQUIRED"
        )


    print()
```

```
    if all_complete:

        print("=" * 60)
        print("ALL SURVEY AREAS COMPLETED")
        print("=" * 60)

        print(
            "Area A: COMPLETE"
        )

        print(
            "Area B: COMPLETE"
        )

        print(
            "Area C: COMPLETE"
        )

        print()
        print(
            "DYNAMIC MULTI-DRONE TASK "
            "REALLOCATION: SUCCESS"
        )

    else:

        print("=" * 60)
        print("MISSION VERIFICATION FAILED")
        print("=" * 60)


    return all_complete


# LAND ALL DRONES


def land_all(self):

    print()
    print("=" * 60)
    print("LANDING ALL DRONES")
    print("=" * 60)


    threads = []


    def land(drone):

        try:

            print(
                f"{drone.name}: LANDING..."
            )

            result = drone.land(
                LAND_SPEED
            )

            print(
                f"{drone.name}: LAND "
                f"{'SUCCESS' if result else 'FAILED'}"
            )

        except Exception as exc:

            print(
                f"{drone.name}: landing error: "
                f"{exc}"
            )


    for drone in self.drones.values():
```

```
        thread = threading.Thread(
            target=land,
            args=(drone,),
            daemon=True
        )

        threads.append(thread)
        thread.start()


    for thread in threads:

        thread.join()


    print()
    print(
        "All landing commands completed."
    )



# SAFETY STOP

def stop_active_behaviors(self):

    for drone in self.drones.values():

        try:

            if drone.go_to.is_running():

                print(
                    f"{drone.name}: stopping "
                    f"active GoTo..."
                )

                drone.go_to.stop()

        except Exception:
            pass



# MAIN EXECUTION

def execute_mission(self):


    # Create drones

    print()
    print("=" * 60)
    print("INITIALIZING DRONE INTERFACES")
    print("=" * 60)


    for name in [
        "drone0",
        "drone1",
        "drone2"
    ]:

        print(
            f"Creating interface: {name}"
        )

        self.drones[name] = SurveyDrone(
            name
        )


    print()
    print(
        "All drone interfaces initialized."
```

```
        )


        # Takeoff

        if not self.takeoff_all():

            return False


        # Start parallel survey

        self.start_initial_surveys()


        # Wait for Drone0 delay

        delay_detected = (
            self.wait_for_delay_detection()
        )

        if not delay_detected:

            return False


        reallocation_success = (
            self.perform_reallocation()
        )

        if not reallocation_success:

            print()
            print(
                "ERROR: Dynamic task "
                "reallocation failed."
            )

            return False


        self.wait_for_remaining_threads()
         mission_success = (
            self.verify_mission()
        )


       # Land

        self.land_all()

        return mission_success


def main():

    rclpy.init()

    manager = AreaSurveyManager()

    success = False

    try:

        success = manager.execute_mission()

    except KeyboardInterrupt:

        print()
        print(
            "Keyboard interrupt received."
```

```
        )

        manager.stop_active_behaviors()

    except Exception as exc:

        print()
        print("=" * 60)
        print("MISSION EXCEPTION")
        print("=" * 60)

        print(exc)

        manager.stop_active_behaviors()

    finally:

        print()
        print("=" * 60)
        print("SHUTTING DOWN DRONE INTERFACES")
        print("=" * 60)

        for drone in manager.drones.values():

            try:
                drone.shutdown()
            except Exception:
                pass

        if rclpy.ok():

            rclpy.shutdown()


    print()

    if success:

        print("=" * 60)
        print("MISSION COMPLETED SUCCESSFULLY")
        print("=" * 60)

    else:

        print("=" * 60)
        print("MISSION DID NOT COMPLETE")
        print("=" * 60)


if __name__ == "__main__":

    main()
```